As a test framework, RSpec fits into a BDD workflow quite well. RSpec helps you "get the words right" and specify exactly what you mean in your tests. You can easily practice the outside-in approach favored in BDD, where you start with acceptance tests and move inward to unit tests.[2] At every level, your expressive tests will guide your software design.

However, RSpec and BDD are not synonymous. You don't have to practice BDD to use RSpec, nor use RSpec to practice BDD. And much of BDD is outside the scope of RSpec; we won't be talking in this book about stakeholder involvement, for instance.

Who We Are

Myron Marston started using RSpec in 2009 and began contributing to it in 2010. He's been its principal maintainer since late 2012. Here are just of the few major improvements he's made to RSpec:

- Composable matchers, which express exactly the pass/fail criteria you need
- `rspec --bisect`, which finds the minimal set of test cases to reproduce a failure
- Integrating RSpec's assertions and mocking libraries with the Minitest framework that ships with Ruby
- The `--only-failures` and `--next-failure` options that let you rerun just your failing tests so that you can fix bugs more quickly

With the insider knowledge Myron provides in this book, you'll learn all of these techniques and more. By the end, you'll be able to get free of just about any problems you run into with your test suite.

Ian Dees stumbled on an old beta of RSpec in 2006. It was just what he needed to build the automated acceptance tests for an embedded touchscreen device. Since then, he's used and taught RSpec for testing everything from tiny microcontrollers to full-featured desktop and web apps.

Who You Are

We hope this book is useful to a wide range of developers, from people who are just getting started with RSpec to those who have written thousands of tests with it. That said, we have made a few assumptions in order to keep the book from getting too bogged down with introductory material.

2. https://dannorth.net/whats-in-a-story/